

BenchmarkAnything: Agent-Driven Construction of Simulator-Ready Microarchitecture Benchmarks

Xiangfeng Sun^{1,†}, Ceyu Xu^{1,†,*}, Chen Bai², and Yuan Xie¹

¹The Hong Kong University of Science and Technology, Hong Kong SAR, China

²Fudan University, Shanghai, China

xsunbv@connect.ust.hk, eeentropy@ust.hk, cbai@fudan.edu.cn, yuanxie@ust.hk

Abstract—The selection of benchmark workloads is of paramount importance in computer architecture, as it establishes the yardstick against which architectural innovations are measured and guided. Yet for decades, the SPEC benchmark suites, comprising merely tens of workloads, have been the de facto standard in academic architectural research, where they are frequently treated as a principal evaluation and optimization target. When a suite this small is relied upon so heavily, it risks architectural overfitting; as our research and prior studies demonstrate, an overly narrow focus can mislead design decisions by overvaluing certain innovations, producing cores that excel on SPEC benchmarks yet underperform on broader, realistic workloads.

To mitigate this overfitting, adopting a large, comprehensive benchmark suite is the natural solution. However, the immense engineering effort required to strip software into the clean, interference-free binary executables demanded by simulators often makes this highly impractical. In this work, we demonstrate that AI agents provide an elegant solution to this challenge. Rather than manually curating yet another static benchmark suite, we introduce an agent-driven workflow capable of autonomously transforming arbitrary open-source repositories into simulator-ready executables. This automated approach makes workload collection highly scalable, allowing us to rapidly harvest hundreds of diverse applications from public repositories into our benchmark suite. Through a comparative analysis of our agent-generated suite against SPEC, we show that it not only achieves higher-fidelity performance assessments but also uncovers novel architectural insights that traditional, static suites fail to expose.

Index Terms—Benchmarking, microarchitecture, workload characterization, LLM agents, computer architecture simulation.

I. INTRODUCTION

Much of computer architecture research begins with bottlenecks revealed by benchmark workloads and ends with evaluation on benchmark suites. In practice, however, architects cannot make design decisions with full visibility into the breadth of real-world deployment scenarios. Human attention is limited, simulation is expensive, and many important production workloads are inaccessible to researchers. As a result, the community relies on curated benchmark suites that aim to represent broad classes of real-world applications in a principled and reproducible manner. Suites such as SPEC therefore serve as essential proxies for hardware research and architectural innovation.

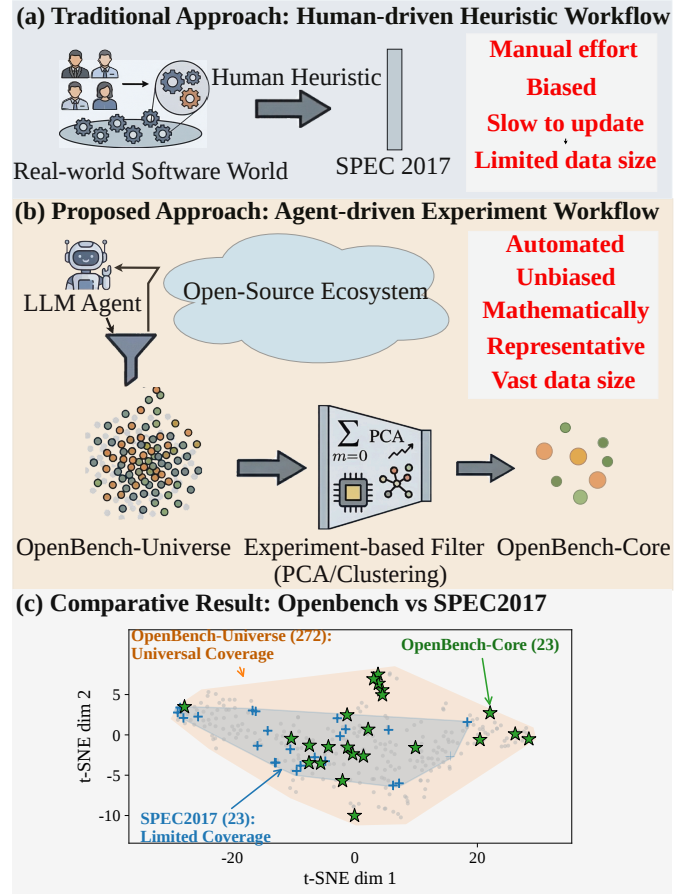


Fig. 1. Comparison of human-driven heuristic workflow and agent-driven experiment workflow. (a) Traditional human heuristic curation is labor intensive and yields limited workload diversity. (b) The agent-driven workflow autonomously gathers applications to form the vast *openbench-universe*. This pool is then statistically distilled into a highly efficient and representative subset named *openbench-core*. (c) Projecting these workloads into the microarchitectural feature space reveals the limited coverage of legacy suites. In contrast, *openbench* spans a much broader behavioral spectrum and better covers contemporary execution bottlenecks.

Because benchmark construction is so consequential, the community has invested decades of expert effort into maintaining recognized benchmark suites that attempt to identify the workloads most representative of real-world use. Yet despite this sustained effort, the combined SPEC family still contains fewer than one hundred benchmark cases across all of its major versions. This limited scale raises a fundamental question for

[†]Xiangfeng Sun and Ceyu Xu contributed equally to this work.

*Corresponding author.

architecture research: can design decisions derived from such a small benchmark set be sufficiently reliable and representative of pervasive real-world workloads?

Prior work has repeatedly shown that SPEC is not fully representative of contemporary real-world workloads and can therefore lead to suboptimal architectural conclusions. Even when SPEC remains informative for certain workload classes, the broader benchmark construction paradigm is still in crisis. The root cause is that benchmark curation has long relied on a human-driven, heuristic-based process. Because the real-world workload space is effectively unbounded while expert attention is inherently limited, benchmark designers must select a small initial candidate pool based largely on subjective judgments of importance and representativeness. This manual selection introduces inherent bias and creates severe representativeness limitations. It is also expensive, labor intensive, and fundamentally difficult to scale. In principle, benchmark construction would benefit from starting from as large and diverse a workload pool as possible and then using a data-centric methodology to distill representative subsets. In practice, however, manual engineering effort sharply limits the size, freshness, and diversity of the candidate pool that can be considered. Moreover, different researchers, companies, and institutions often care about different workload mixes and deployment scenarios. A single static benchmark suite cannot adapt to these varying needs, nor can it evolve fast enough to track the rapid emergence of new applications and software stacks. Taken together, these limitations indicate that the field needs a paradigm shift in benchmarking toward methodologies that are more representative, less biased, more scalable, and more transferable across use cases.

To overcome the fundamental limitations of the human-driven, heuristic-based benchmark construction paradigm, we propose a shift toward an agent-driven, data-centric benchmark construction methodology that is more representative, less biased, more scalable, and more transferable across use cases, as shown in Figure 1. Specifically, we introduce *BenchmarkAnything*, a fully automated framework designed to improve workload representativeness through large-scale workload collection and statistical distillation. This methodology operates in two phases. In the first phase, an LLM-driven agent flow autonomously navigates open-source platforms to collect candidate applications, resolve build dependencies, and compile them into executable artifacts. This automated collection process reduces human selection bias and scales with the pace of software evolution. In the second phase, the framework executes the collected applications to extract a broad set of microarchitectural features. It then applies rigorous experimental and statistical analysis to systematically distill this large candidate pool into a benchmark suite with strong empirical representativeness over the collected workload space.

Based on this agent-driven, data-centric benchmark construction methodology, we construct the *openbench* benchmark family. It includes *openbench-universe*, a large benchmark suite that bridges industry-standard tasks with cutting-edge software deployments, and *openbench-core*, a distilled subset for efficient simulation. *openbench-universe* consists of 272

benchmarks spanning 14 domains, providing substantial diversity and coverage. Our comprehensive microarchitectural characterization (IV) shows that *openbench* achieves higher fidelity than legacy alternatives. By reducing manual selection bias and grounding benchmark selection in experimental validation, *openbench* more faithfully captures the memory-hierarchy pressure and instruction footprints of contemporary workloads [1]–[3]. As a result, it exposes critical architectural bottlenecks that legacy benchmark suites often fail to trigger. Overall, *openbench* provides a more realistic and timely evaluation target for the design of next-generation high-performance processors. Our aim is to *complement* rather than replace SPEC: SPEC remains a carefully curated, stable, high-quality reference, while *BenchmarkAnything* contributes a fast, low-cost, and portable way to keep evaluation aligned with software as it evolves. Because the framework is retargetable, the same flow can convert a research group’s or a company’s own important applications into a simulator-ready suite, rather than committing every user to a single fixed set of workloads.

In summary, this paper makes the following contributions.

- We introduce an agent-driven, data-centric benchmark construction paradigm that replaces traditional manual heuristics with large-scale automated workload collection and statistical distillation.
- We present *BenchmarkAnything*, a fully automated framework that can autonomously search open-source platforms, resolve build dependencies, and produce simulator-ready benchmark artifacts.
- We construct the *openbench* benchmark family, including its universe and core subsets, and show that it exposes modern execution bottlenecks that are largely absent in SPEC CPU2017.
- We show that this workflow substantially broadens workload diversity and coverage while preserving simulation efficiency through principled core-subset selection.

II. BACKGROUND AND MOTIVATION

A. Benchmark Importance for CPU Design

Benchmarks are a primary feedback proxy between real software and CPU design decisions. Because processor performance depends jointly on microarchitecture and workload characteristics, architects rely on benchmark suites to approximate the target deployment space during pre-silicon design, post-silicon evaluation, and subsequent tuning of compilers and runtimes [4], [5]. A well-designed suite provides a common, repeatable basis to compare alternative designs, quantify bottlenecks, and reason about tradeoffs in performance, power, and area [6], [7]. The choice and construction of benchmarks therefore strongly affect the validity of design conclusions. Navigating this space, however, is fundamentally bottlenecked by simulation overhead: simulating very large application sets is infeasible, so architects must select representative benchmark and input pairs.

B. Existing CPU Benchmark and Manual Benchmark Curation

Standardized suites such as SPEC CPU have been central to microarchitecture research and industrial CPU evaluation

for more than three decades [1], [8], [9]. SPEC suites stress the processor core, memory hierarchy, and compiler toolchain under a controlled methodology, and have become the de facto basis for comparing commercial systems and validating academic proposals. The latest release, SPEC CPU2017, extends CPU2006 with larger and more complex integer, floating-point, and multi-threaded workloads to better reflect contemporary software [1], [10], [11]. All of these suites rely on significant manual curation: selecting applications, defining build procedures, resolving dependencies, and engineering representative inputs under strict runtime and resource constraints.

Despite their impact, existing CPU benchmarks and manual curation exhibit several limitations. First, SPEC CPU2017 shows substantial internal self-similarity and does not fully capture emerging workload classes. Performance counter and clustering studies report strong redundancy and show that most CPU2017 workloads occupy a narrow region of the behavior space, while many modern data center, graph analytics, and EDA workloads exhibit distinct characteristics [4], [11], [12]. Memory hierarchy studies find that almost half of SPEC CPU2006 and CPU2017 benchmarks have very low last-level cache miss ratios on contemporary processors, and that CPU2017 places even less pressure on the memory hierarchy than CPU2006 [1], [2]. Traditional CPU suites also show lower L1 instruction miss rates and higher instruction locality than big data and internet service workloads, limiting their ability to stress front-end structures and large caches realistically [3]. SPEC-based evaluations may therefore systematically under-represent the instruction footprints, working set sizes, and microarchitectural bottlenecks that dominate modern cloud and data-intensive software.

Second, manual curation does not scale with the size and dynamism of modern software ecosystems. The creation of SPEC CPU2017 required more than a decade, and the suite still ships with only a small number of official input sets per benchmark [13]. Amaral et al. note that the lack of multiple workloads per application complicates evaluation of techniques that depend on workload variation, and that constructing additional workloads demands substantial manual effort and deep application knowledge. Similar concerns arise in achieving consistent and comparable CPU2017 evaluation across configurations, where arbitrary specificity in some parameters and ambiguity in others lead to large performance variability and unfair comparisons [14]. More broadly, benchmarking and benchmark maintenance are increasingly recognized as labor-intensive, error-prone processes that require careful workflow management and automation to remain reproducible and extensible [15], [16]. Crucially, the persistent lack of a standardized, automated methodology prevents the community from scaling benchmark suites to match the rapid evolution of modern software.

C. LLM Agents in Software Engineering

Recent advances in large language models have catalyzed a shift from passive code models to LLM-based agents that perceive context, invoke tools, and autonomously plan multi-step workflows for software engineering tasks [17]–[22]. Compared with single shot LLM usage, these agents aim to cover broader

portions of the software development lifecycle and to behave more like collaborative team members than isolated prediction engines [23], [24].

Surveys now distinguish three layers of capability. First, LLMs as code models support code completion, translation, summarization, and repair [25]–[28]. Second, LLM-based agents wrap LLMs with perception, memory, and action modules to enable tool use, repository-level reasoning, and iterative refinement over long contexts. Third, multi-agent systems coordinate specialized agents that play roles such as designer, coder, tester, and reviewer [23]. Systematic reviews report rapid growth of this line of work across requirements engineering, code generation, test generation, maintenance, and DevOps [19], [29]. Recent open-source systems such as CLI-Anything make this shift concrete by transforming arbitrary repositories into agent-native CLIs with structured outputs and stable command interfaces [30]. Our aim of abstraction is different: rather than turning anything into a CLI, we turn diverse real-world software into benchmarks, a promising direction for addressing this long-standing scalability crisis.

III. BENCHMARKANYTHING: AGENT-DRIVEN FRAMEWORK

Our key insight is that *any* open-source software repository can be automatically transformed into a standard benchmark through a sufficiently capable autonomous agent workflow. The open-source ecosystem continuously generates software that reflects contemporary computing patterns, yet manual curation cannot harvest this diversity at scale. To bridge this gap, we propose *BenchmarkAnything*, an automated framework that leverages LLM-based agents to autonomously retrieve, compile, verify, and standardize benchmark applications from diverse open-source repositories. Figure 2 presents the end-to-end workflow, which comprises four phases: benchmark searching, autonomous build and workload synthesis within an isolated sandbox, host-environment verification through microarchitectural simulation, and standardized artifact collection. In our current implementation, 93% of the open-source projects retrieved in Phase 1 are successfully transformed into standardized benchmarks by this end-to-end flow. All agents are driven by a single backbone model, MiniMax-M2.5, which balances cost efficiency, throughput, and coding proficiency. The remainder of this section describes each phase and its design rationale.

A. Phase 1: Benchmark Searching

The open-source ecosystem hosts millions of repositories spanning diverse languages, build systems, and application domains, so identifying candidates requires a search strategy that is both broad and systematic. As illustrated in Figure 2(a), the first phase analyzes the user-provided benchmark requirements. A primary scheduling agent classifies them into distinct categories and dispatches each category to a dedicated subagent. The categories encode *how* a candidate should be sought. **Type A** requirements are domain- or algorithm-driven, such as graph analytics or sparse linear algebra. **Type B**

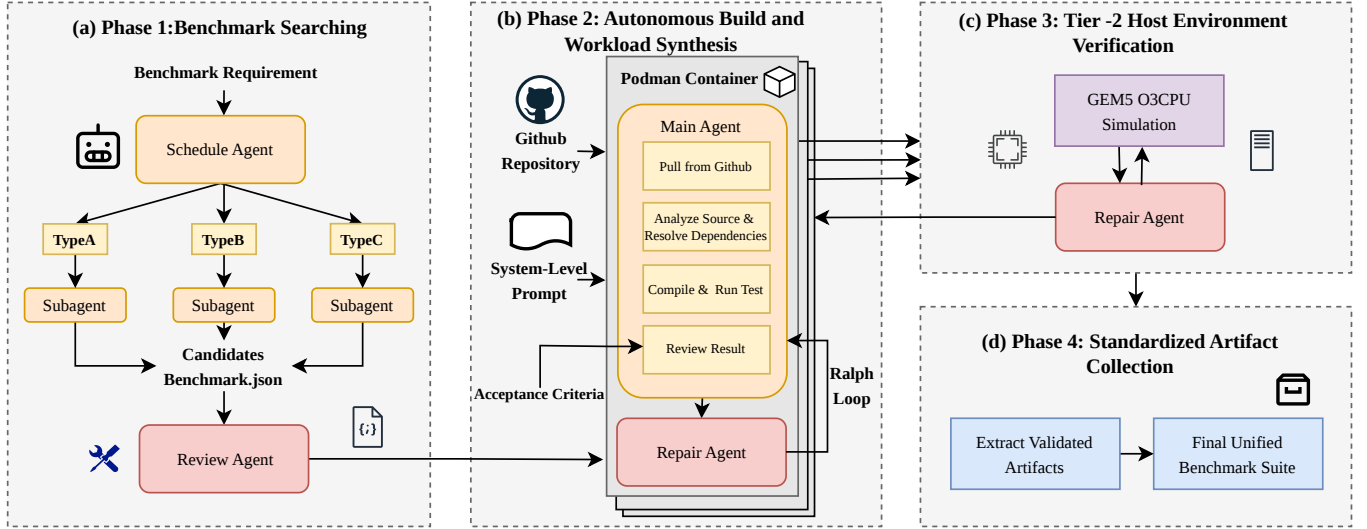


Fig. 2. Overview of the *BenchmarkAnything* framework. (a) Phase 1, Benchmark Searching. A scheduling agent classifies benchmark requirements into Type A, Type B, and Type C, and dispatches dedicated subagents that retrieve candidate repositories from open-source platforms in parallel. (b) Phase 2, Autonomous Build and Workload Synthesis. Within an isolated Podman sandbox, the main agent resolves software dependencies and performs architecture-aware compilation. Compilation or execution failures trigger the Ralph loop, an iterative reflection-and-repair cycle where a repair agent diagnoses errors and supplies targeted feedback until the benchmark satisfies strict static-linking, execution, and output acceptance criteria. (c) Phase 3, Tier-2 Host-Environment Verification. Compiled artifacts undergo microarchitectural compatibility validation through gem5 simulation. Simulation failures first invoke a repair agent for targeted fixes; unresolvable issues are routed back to the container for recompilation. (d) Phase 4, Standardized Artifact Collection. The framework extracts validated binaries, wrapper scripts, and input data, and reorganizes them into a uniformly formatted, simulation-ready benchmark suite.

requirements are adoption-driven and target widely used, actively maintained projects such as high-star repositories or packages with large install bases. **Type C** requirements are capability- or keyword-driven, such as a compression codec or a regular-expression engine. A single user request typically expands into a mixture of all three categories, so the candidate pool spans both flagship applications and the long tail of the ecosystem. This parallelized decomposition is deliberate: different domains reside in distinct repository ecosystems with varying naming conventions and metadata structures, and specialized subagents apply domain-appropriate search heuristics more effectively than a single monolithic query.

Search and review are governed by explicit, hardware-aware selection criteria. Each candidate is scored along four axes. *Adoption and relevance* captures repository popularity, release maturity, active maintenance, and documentation quality. *CPU-centric executability* requires a real application entry point that performs substantive computation on the CPU. *Input availability* prefers official or repository-provided inputs and documented public datasets. *Simulator compatibility* requires the workload to be admissible under the target simulator. The review agent additionally enforces an exclusion filter: repositories that are GPU-only, accelerator-only, or expressed purely in CUDA/OpenCL/HLS, as well as projects containing only unit tests, micro-demos, or build scaffolding, are rejected unless an explicit CPU implementation is targeted. The retrieved results are aggregated into a unified `CandidatesBenchmark.json` manifest, which a review agent evaluates to eliminate duplicate and non-compliant entries. If the number of valid candidates falls below the target threshold, the review agent instructs the search agent to retrieve more. In practice the search agent deliberately over-provisions candidates, because collecting a large initial pool and filtering offline is far more efficient

than issuing repeated narrowly scoped queries, so secondary searches are rarely needed. Once validated, the manifest is forwarded to Phase 2.

```
[Role] Autonomous executor for BenchmarkAnything with
      sudo access.

[Step 1] Fetch source code from GitHub.
[Step 2] Compile with STRICT rules:
- Install missing deps autonomously.
- MUST enforce static linking.
- Use '-march=x86-64'. PROHIBIT AVX/AVX2/AVX-512 to
  ensure gem5 SE compatibility.
...
[Step 3] Prepare Inputs & Run Script:
- Use official or repository-provided inputs.
- Generate 'run.sh'. No infinite loops or sleep
  commands.
...
[Acceptance Criteria]
1. Binary is a statically linked ELF.
2. Native execution succeeds (exit code 0).
3. Retires enough instructions to fill the 1B-insn
  warmup + 100M-insn ROI window.
```

Listing 1. Extracted rules and constraints from the *main agent* prompt.

B. Phase 2: Autonomous Build and Workload Synthesis

Compiling arbitrary open-source software is inherently unpredictable. Each repository may depend on a unique combination of libraries, toolchains, and system utilities, and resolving these dependencies on a shared host risks polluting the build environment of subsequent candidates. Figure 2(b) illustrates how the second phase addresses this challenge. The framework employs Podman to instantiate an isolated container for each candidate, eliminating the need for superuser privileges on the host while enabling safe resolution of heterogeneous dependencies. Within the container, the agent is granted elevated privileges strictly for invoking package managers and

installing missing libraries. A system-level prompt governs compilation, enforcing output requirements such as a statically linked x86-64 executable, because statically linked binaries are a strict prerequisite for simulated execution environments that lack dynamic linker support.

The main agent drives the core build process. It pulls the source code, analyzes the project documentation to resolve dependencies, and performs architecture-aware compilation that disables unsupported vector extensions such as AVX-512 and enforces single-threaded execution. The agent then provisions input datasets following a strict provenance hierarchy: it first prefers official or repository-provided inputs and the project’s own example workloads, falling back to documented public datasets referenced by the project. The chosen source and its provenance are recorded in the benchmark manifest, and when a binary exposes distinct behavior under different inputs the agent may retain multiple input configurations. Each input is sized so that the workload retires enough instructions to fill the subsequent 1B-instruction warmup and the full 100M-instruction region of interest, guaranteeing a measurable steady-state window rather than a setup-dominated run. After initial native execution tests, the framework evaluates the result against three formal acceptance criteria. If any criterion fails, the workload enters the Ralph loop, an iterative reflection-and-repair mechanism connecting the repair agent and the main agent. In each iteration, the repair agent diagnoses compilation logs or runtime errors and provides targeted feedback, enabling the main agent to adjust dependencies, compilation flags, or input configurations. The cycle continues until all criteria hold or a configurable iteration budget is exhausted. Three criteria gate the exit: the binary must be a strictly statically linked ELF executable, it must run through a standardized shell script without crash signals such as segmentation faults, and it must retire enough instructions natively to fill the 1B-instruction warmup and 100M-instruction region of interest, a window whose completion is then confirmed under `gem5` in Phase 3. Workloads that satisfy all three proceed to the next phase.

C. Phase 3: Tier-2 Host Environment Verification

Passing the intra-container criteria ensures the benchmark works as a native executable but does not guarantee compatibility with a detailed microarchitectural simulator. System calls, memory-mapped I/O, and complex vector instructions that work on native hardware may cause fatal errors in simulation, so a second verification tier is essential. As illustrated in Figure 2(c), the compiled artifacts cross the isolation boundary into the host environment, where the framework executes each workload under a `gem5` O3CPU simulation. If the simulation fails due to unsupported system calls or unrecognized instructions, a repair agent first applies targeted fixes directly to the execution artifacts. If the repair agent cannot resolve the issue, a fallback strategy routes the workload back to the main agent in the container for complete recompilation under stricter architectural constraints.

D. Phase 4: Standardized Artifact Collection

The final phase, shown in Figure 2(d), addresses a persistent barrier to benchmark adoption. When researchers manually

port open-source workloads for simulation, they must contend with heterogeneous build artifacts, inconsistent input formats, and undocumented execution scripts, an overhead that often discourages adopting new workloads entirely. Our framework instead extracts only the validated artifacts from the sandbox, discarding intermediate build files and isolating the core functional components: the statically linked binary, the execution wrapper script, and any required input data. By enforcing this uniform directory topology across all applications, the framework yields a plug-and-play suite that deploys seamlessly in subsequent architectural experiments. This phase also makes the suite reproducible despite LLM nondeterminism. *BenchmarkAnything* separates *discovery* from *replay*: agent decisions may differ across runs, but each accepted benchmark freezes the source revision, build commands and flags, input files and provenance, execution command, validation checks, and `gem5` invocation. The published package can therefore be replayed without invoking a model and remains fixed even if discovery yields another candidate universe.

E. The Ralph Loop: Iterative Reflection and Repair

Among the four phases, the Ralph loop is the single most critical mechanism that enables *BenchmarkAnything* to operate at scale without human intervention. The Ralph loop is a general agentic pattern in which an agent is repeatedly re-prompted against a fixed specification until it succeeds, originally articulated by Geoffrey Huntley [31] and since widely adopted in agentic coding frameworks. Our contribution is not the loop itself but its specialization to benchmark construction: a formal, simulator-aware acceptance gate comprising static-linking, crash-free execution, and region-of-interest completion checks that decides when an iteration may exit.

The challenge is fundamental. Unlike curated suites whose build procedures are hand-tuned over months, each collected repository presents a unique combination of build-system conventions, dependency graphs, and implicit environmental assumptions, so a single-pass compilation strategy would fail on a large fraction of candidates. The Ralph loop structures the build process as a closed-loop control system in which failures drive iterative refinement rather than immediate rejection. In practice, static linking is the dominant engineering obstacle: many modern projects assume dynamic linking or pull in components such as NSS, network, or threading libraries that resist a fully static build. Substituting static-friendly libraries, stubbing optional features, and pruning irreducibly dynamic dependencies constitute the principal repair tasks the loop automates, and applications that cannot be reduced to a self-contained static binary are deferred by the acceptance gate.

Two properties deserve emphasis. First, the acceptance criteria in Listing 1 serve as a formal verification gate rather than a soft heuristic, ensuring that every benchmark entering subsequent phases is simulation-ready by construction. Second, the iterative repair cycle mirrors the debugging workflow of a human developer who reads error messages, hypothesizes a fix, and retries, but operates at machine speed and scales to hundreds of repositories in parallel.

IV. *openbench* CHARACTERIZATION VS. SPEC CPU2017

A. Experimental Setup and Profiling Space

To characterize microarchitectural behaviors rigorously, we evaluate 30 unique configurations detailed in Table I. This multidimensional design space encompasses commercial processors ranging from ultra-low-power embedded controllers to aggressive server-class out-of-order engines, and we prune infeasible design points to maintain architectural realism: pairing a wide 10-instruction fetch width with a simple in-order core, for example, is unbalanced and unrealistic. We scale execution pipelines by varying fetch widths from one to eight instructions per cycle across in-order and out-of-order models, and scale the reorder buffer from 64 to 384 entries with proportional issue queues to capture the diminishing returns of backend expansion. We stress application working sets by varying L1 caches from 8kB to 64kB and L2 caches from 64kB to 2MB, instantiate TAGE, Tournament, BiMode, and Local branch predictors to evaluate frontend control-flow sensitivity, and pair Signature Path and Stride prefetchers with DRRIIP, SHIP, BRRIP, and LRU replacement policies to expose heterogeneous memory access patterns. We execute this profiling space with the *gem5* simulator in Syscall Emulation mode under the x86 ISA. To avoid distortion from unrepresentative setup phases, we fast-forward the first 1B instructions of each benchmark as warmup on a simple CPU with warm caches, then reset all statistics and collect them over a precisely defined 100M-instruction Region of Interest (ROI) executed on the detailed out-of-order core. Both the 1B warmup and the 100M ROI are user-adjustable instruction counts of our flow and follow common practice in simulation studies [32]. Because any microarchitecture-dependent characterization can be biased by the chosen machine configurations [33], our selection is deliberately spread across the principal axes, namely core type, pipeline width, reorder-buffer and load/store-queue depth, L1/L2 capacity, and the choice of branch predictor, prefetcher, and replacement policy, so the 30 points sample balanced, realizable designs rather than a dense grid of one structure in isolation. This many-parameter, machine-dependent sweep complements two established methodologies: statistically rigorous design-of-experiment sampling such as Plackett–Burman, which identifies a minimal set of configurations to probe a large space [34], and broad multi-ISA and compiler similarity studies of benchmark suites [35].

B. The Scale, Diversity, and Coverage of the *openbench-universe*

Equipped with these configurations, we evaluate the workload pool collected by *BenchmarkAnything*, a superset we denote *openbench-universe* containing 272 standardized applications from diverse open-source projects. Because *BenchmarkAnything* can continuously collect more benchmarks, we focus on 272 for tractability. To make the suite transparent and auditable, Table II lists workloads in each application domain. The 272 benchmarks will be open-sourced with the artifact when the paper is accepted.

As shown in Figure 3, SPEC2017 concentrates on a narrow set of traditional domains, whereas the 272 workloads in

TABLE I

MICROARCHITECTURE CONFIGURATIONS USED IN THE EVALUATION. WE SYSTEMATICALLY EXPLORED VARIOUS ARCHITECTURAL CONFIGURATIONS AND PRUNED INFEASIBLE DESIGN POINTS TO COVER A REPRESENTATIVE RANGE OF CONTEMPORARY SINGLE-CORE CPU MICROARCHITECTURES. PIPELINE WIDTH REFERS TO THE UNIFIED FETCH/DECODE/ISSUE/COMMIT WIDTH. FOR IN-ORDER CORES, ROB AND IQ ARE NOT APPLICABLE (-). L1 CACHES ARE SYMMETRIC IN SIZE AND ASSOCIATIVITY (I/D). PREFETCHERS, BRANCH PREDICTORS (BP), AND REPLACEMENT (REPL) POLICIES LEVERAGE BUILT-IN GEM5 MODELS.

Cfg	Core	Width	ROB	IQ	LQ/SQ	L1 (I/D)	L2	BP	Prefetch	Repl
C1	OoO	8	384	128	96/72	64kB/8-way	2MB/16-way	TAGE	SPP	DRRIIP
C2	OoO	8	384	128	96/72	64kB/8-way	2MB/16-way	TAGE	Stride	SHIP
C3	OoO	8	256	97	72/56	32kB/8-way	1MB/16-way	TAGE	Stride	BRRIP
C4	OoO	8	384	128	96/72	64kB/8-way	256kB/8-way	TAGE	None	LRU
C5	OoO	8	384	128	96/72	64kB/8-way	2MB/16-way	Local	Stride	DRRIIP
C6	OoO	6	256	97	72/56	64kB/8-way	2MB/16-way	TAGE	SPP	SHIP
C7	OoO	6	192	80	56/42	32kB/8-way	1MB/16-way	Tourn.	Stride	DRRIIP
C8	OoO	6	192	80	56/42	32kB/8-way	1MB/16-way	TAGE	Stride	TreePLRU
C9	OoO	6	256	97	72/56	16kB/4-way	2MB/16-way	TAGE	None	BRRIP
C10	OoO	6	128	64	48/36	64kB/8-way	1MB/16-way	Tourn.	Stride	LRU
C11	OoO	4	192	80	64/48	64kB/8-way	2MB/16-way	TAGE	SPP	SHIP
C12	OoO	4	192	80	64/48	32kB/8-way	1MB/16-way	TAGE	Stride	DRRIIP
C13	OoO	4	128	64	48/36	32kB/8-way	512kB/8-way	Tourn.	Stride	TreePLRU
C14	OoO	4	128	64	48/36	32kB/8-way	512kB/8-way	BiMode	Stride	LRU
C15	OoO	4	96	48	40/32	32kB/8-way	256kB/8-way	Tourn.	None	LRU
C16	OoO	4	192	80	64/48	64kB/8-way	1MB/16-way	Local	Stride	BRRIP
C17	OoO	4	128	64	48/36	16kB/4-way	512kB/8-way	TAGE	Stride	Random
C18	OoO	4	128	64	48/36	64kB/8-way	2MB/16-way	BiMode	None	SHIP
C19	OoO	3	96	48	40/32	32kB/8-way	512kB/8-way	Tourn.	Stride	LRU
C20	OoO	3	96	48	40/32	16kB/4-way	256kB/8-way	BiMode	None	TreePLRU
C21	OoO	2	96	48	32/24	32kB/8-way	512kB/8-way	BiMode	Stride	LRU
C22	OoO	2	64	32	24/20	32kB/8-way	256kB/8-way	BiMode	None	LRU
C23	OoO	2	64	32	24/20	16kB/4-way	128kB/8-way	Local	None	FIFO
C24	OoO	2	128	64	48/36	64kB/8-way	1MB/16-way	TAGE	SPP	DRRIIP
C25	InO	2	-	-	-/-	64kB/8-way	1MB/16-way	Tourn.	Stride	LRU
C26	InO	2	-	-	-/-	32kB/4-way	512kB/8-way	BiMode	Stride	LRU
C27	InO	2	-	-	-/-	16kB/4-way	256kB/8-way	Local	None	LRU
C28	InO	1	-	-	-/-	32kB/4-way	256kB/8-way	BiMode	None	LRU
C29	InO	1	-	-	-/-	16kB/4-way	128kB/8-way	Local	None	FIFO
C30	InO	1	-	-	-/-	8kB/2-way	64kB/4-way	None	None	Random

openbench-universe cover a drastically wider variety of modern domains including machine learning, graph processing, and cryptography. Moreover, whereas all 23 SPEC CPU2017 benchmarks are written in legacy languages such as C, C++, and Fortran, *openbench-universe* incorporates applications written in Rust, Go, and other modern systems languages, capturing the distinct execution characteristics of contemporary software stacks.

Beyond application-level breadth, Figure 4 reveals the *microarchitectural* diversity of *openbench-universe*. We represent each workload as a 30-dimensional IPC feature vector spanning C1–C30, project it to two dimensions via t-SNE, and apply K-Means clustering with $k=10$. Rather than labeling clusters by application domain, we assign each an intuitive *behavioral archetype* from its aggregate microarchitectural profile, including mean IPC, cache miss rates, and configuration sensitivity.

The projection confirms that *openbench* workloads do not collapse into a monolithic group but disperse into ten well-separated clusters spanning the full spectrum of hardware bottlenecks. At one extreme lies the *Memory Bound* cluster, whose members achieve the lowest IPC due to severe data-cache pressure. At the other extreme, the *High IPC (I-Cache Bound)* cluster sustains the highest throughput despite elevated instruction-fetch miss rates. Between these poles, *D-Cache Bound* and *Low IPC (Backend Limit)* workloads expose distinct memory-hierarchy pathologies, while *Hardware Insensitive* benchmarks remain indifferent to configuration changes. The *Highly Sensitive to Hardware* and *Hardware Responsive* clusters exhibit pronounced IPC variation across

TABLE II

OVERVIEW OF THE *openbench-universe*: 272 AUTOMATICALLY
CONSTRUCTED BENCHMARKS GROUPED INTO 14 APPLICATION DOMAINS.

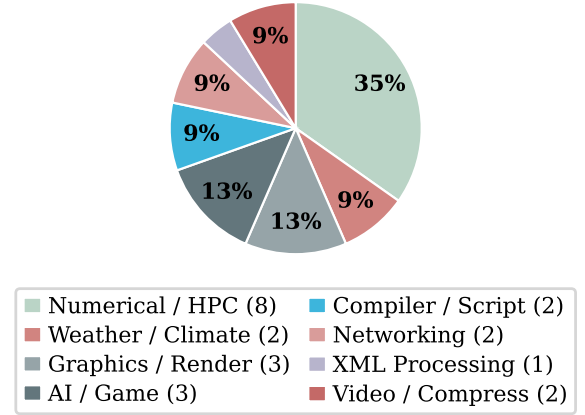
Domain	Benchmarks
Numerical / HPC (54)	OpenBLAS, FFTW3, blis, GSL, flint, sundials, zfp, ...
Cryptography (26)	mbedtls, botan, cryptopp, bearssl, Nettle, libtom-crypt, BLAKE2, monocrypt, hacl-star, XKCP, ...
Miscellaneous (25)	hyperscan, QuantLib, SEAL, primesieve, TAlib, ...
System / Embedded (20)	sqlite, fio, mimalloc, jemalloc, tcmalloc, busybox, ...
Parsing / String (20)	pcr2, HarfBuzz, rapidjson, protobuf-c, pugixml, utf8proc, cmark, snowball, ...
Data Structures (18)	CRoaring, marisa-trie, judy, ...
Graphics / Sim (18)	bullet3, box2d-v3, pbrt-v4, appleseed, meshoptimizer, reactphys3d, ...
Compression (16)	brotli, fast-lzma2, lzfs, miniz, xxHash, streamvbyte, fzip, ...
Sorting (16)	pdqsort, timsort, ska_sort, ...
Interp. / Compiler (15)	lua, luau, mruby, duktape, jerryscript, chibi-scheme, asmcjit, re2c, rv32emu, ...
AI / Game (11)	Stockfish, Fairy-SF, cadical, picosat, ...
Graph / Search (11)	igraph, snap, lemon, ...
Bioinformatics (11)	minimap2, htlib, kallisto, seqan, edlib, parasail, WFA2-lib, seqtk, ...
Media / Signal (11)	libwebp, libtiff, lame, giflib, speexdsp, ...

the 30 configurations, making them valuable for stress-testing pipeline and cache trade-offs. The *I-Cache Bound* cluster shares elevated instruction-cache miss rates with its High IPC counterpart yet operates at moderate throughput, isolating pure frontend pressure, and the *High IPC Compute* cluster represents workloads that are primarily compute-limited. The largest cluster, *Balanced*, anchors the center of the embedding with moderate, well-distributed resource demands.

Every cluster carries a unique label because the ten archetypes derive from multi-dimensional profiles rather than any single scalar metric: two clusters sharing a similar mean IPC still diverge along orthogonal axes such as cache-miss distribution and configuration sensitivity. By exposing this broad behavioral landscape, *openbench* gives architects the bounds to stress-test robust processor designs against the diversity of modern software.

To further analyze the difference between *openbench-universe* and SPEC2017, Figure 5 plots the *openbench* geometric-mean IPC against the SPEC2017 geometric-mean IPC across all 30 configurations. The strong Pearson correlation of $r = 0.983$ confirms suite-level consistency, yet notable outliers expose configuration-specific divergences, and Figure 5(b) zooms into the two most off-trend configurations. Configuration C9, a 6-wide design without a prefetcher, achieves a disproportionately high *openbench* IPC relative to its SPEC2017 score because the absent prefetcher penalizes SPEC2017's cache-sensitive workloads more severely while

(a) SPEC2017 (23)



(b) OpenBench (272)

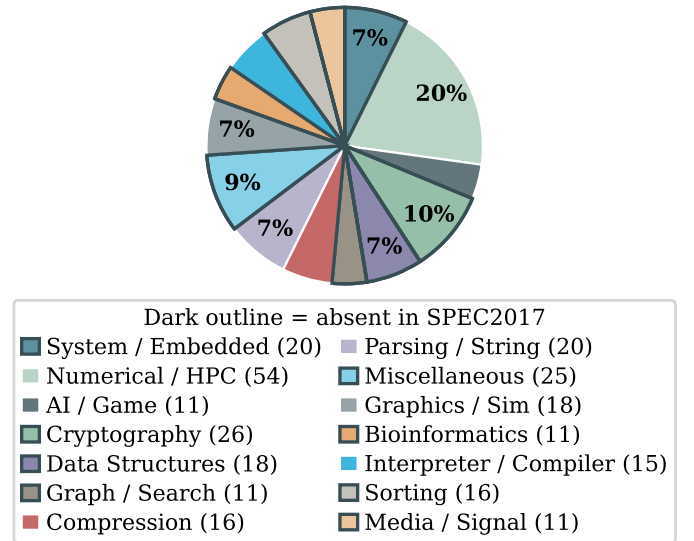


Fig. 3. Application-domain distribution of *openbench-universe* and SPEC CPU2017. The pie chart illustrates that *openbench-universe* encompasses a significantly broader range of modern application domains than SPEC CPU2017, reflecting contemporary software ecosystems.

openbench's diverse applications are comparatively resilient. Conversely, C6, which pairs a 64 kB L1 with an SPP prefetcher and SHiP replacement, falls below the regression line: its advanced cache-hierarchy optimizations substantially boost SPEC2017's regular access patterns but provide much smaller gains on *openbench*'s heterogeneous mix. Together, these two points indicate that modern prefetcher and cache-replacement designs can look stronger on SPEC2017 than on broader real workloads, suggesting a degree of overfitting to SPEC2017 that *openbench* makes visible.

Beyond individual outliers, Figure 5(c) illustrates a systemic design trade-off by zooming into C24 and C15. C24 is a 2-wide core with a heavily provisioned memory subsystem: a deep 128-entry reorder buffer, a large 1 MB L2, and an aggressive Signature Path Prefetcher with DRRIP replacement. C15 is a wider 4-wide core with a lean memory subsystem: a shallow 96-entry reorder buffer, a compact 256 kB L2, and no hardware prefetcher. Under SPEC2017, C24 and C15 achieve parity. The *openbench-universe* evaluation instead reveals an

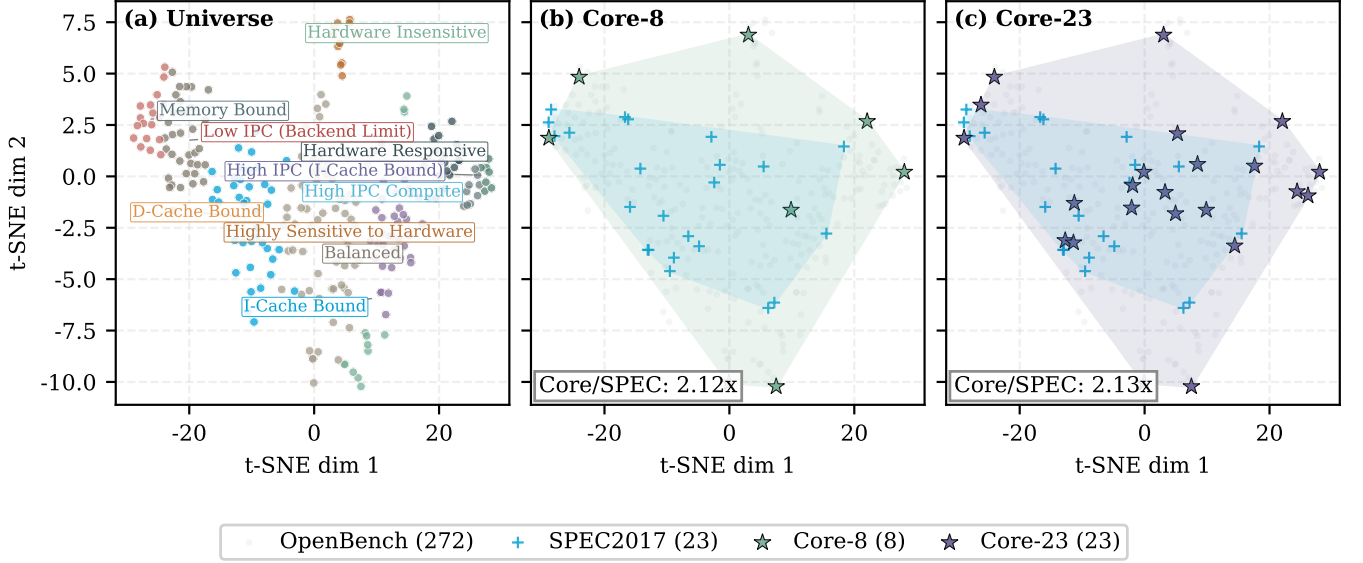


Fig. 4. Unified t-SNE projection of the *openbench* workload space across C1–C30, illustrating the relationship between the full universe and its distilled subsets. All three panels share the same embedding. (a) The full 272-workload *openbench-universe* grouped into 10 behavioral clusters via K-Means. Each cluster is annotated with an intuitive microarchitectural archetype, ranging from *Memory Bound* to *High IPC (I-Cache Bound)*, reflecting its dominant hardware bottleneck. (b) *openbench-core* Core-8 (green stars) versus SPEC2017 (cyan crosses). The Core-8 convex hull ($2.12\times$ SPEC hull area) substantially exceeds the SPEC2017 footprint with only eight benchmarks. (c) *openbench-core* Core-23 (purple stars) versus SPEC2017 (cyan crosses). Core-23 is a *nested superset* of Core-8 ($\text{Core-8} \subset \text{Core-23}$), so its hull provably contains the Core-8 hull; it reaches $2.13\times$ the SPEC hull area while further improving fidelity to the full universe.

inversion where the lean-memory C15 significantly outperforms C24: on modern applications with massive code footprints, intricate branch behavior, and irregular memory access, C15’s wider pipeline sustains higher throughput while C24’s deep memory resources stay underutilized amid frequent stalls. C15 and C24 are deliberately unbalanced corner designs; the point is not that C15 is universally better, but that benchmark choice alone can flip the apparent ranking of two candidates. Relying solely on SPEC2017 can thus obscure meaningful architectural differences during design-space exploration, and complementing it with *openbench* restores visibility into these distinctions.

To evaluate the overhead of constructing *openbench-universe*, we analyze token usage. Figure 6 shows the distribution: most benchmarks fall below 0.05M tokens, with a mean of 0.03M and a median of 0.021M, demonstrating the extremely low cost and scalability of *BenchmarkAnything*.

C. Experiment-based Selection from *openbench-universe* to *openbench-core*

While *openbench-universe* provides unprecedented scale, practical architectural simulation demands compact, non-redundant suites. We therefore apply a principled statistical distillation to reduce the 272 raw workloads into a concise representative suite, denoted *openbench-core*.

We represent each benchmark as a 30-dimensional IPC feature vector spanning C1–C30 and apply PCA to project this space onto its principal components. Figure 7 shows the elbow analysis that determines the optimal subset size: k-medoids clustering in the PCA feature space reveals a clear inflection at $k = 8$, providing a strong trade-off between representational fidelity and simulation cost.

With k determined, we run k-medoids on the Pearson distance matrix in PCA space to select one representative medoid per cluster. We adopt Pearson rather than Euclidean distance because it measures the similarity of IPC *profiles* across configurations, capturing how a benchmark responds to microarchitectural changes irrespective of absolute throughput. Two workloads with identical scaling trends but different absolute IPC stress the same hardware structures in the same relative way and should share a cluster, whereas Euclidean distance would conflate magnitude differences with profile-shape differences. Each medoid is an actual benchmark whose per-configuration IPC profile lies closest to its cluster centroid in the reduced feature space. To prevent the selection of obscure micro-benchmarks, we augment the distance-based objective with a soft tie-breaking prior that favors widely adopted projects.

Table III lists the resulting Core-8 subset in *openbench-core*. Each row corresponds to one medoid retained from a distinct behavioral cluster, and the justification column summarizes the dominant property that made that benchmark the representative of its cluster. The eight benchmarks also span eight distinct application domains, from memory-latency stress tests and hash-intensive data structures to bioinformatics alignment and physics simulation, ensuring broad algorithmic diversity in addition to microarchitectural coverage. To prove that it is a representative subset of *openbench-universe*, we validate Core-8 along two complementary axes:

Ranking fidelity. Figure 8 compares configuration rankings from Core-8 against the full universe, both ordered by geometric-mean IPC. The Spearman coefficient $\rho = 0.989$ confirms that design decisions based on the eight-benchmark subset closely mirror those from all 272 workloads.

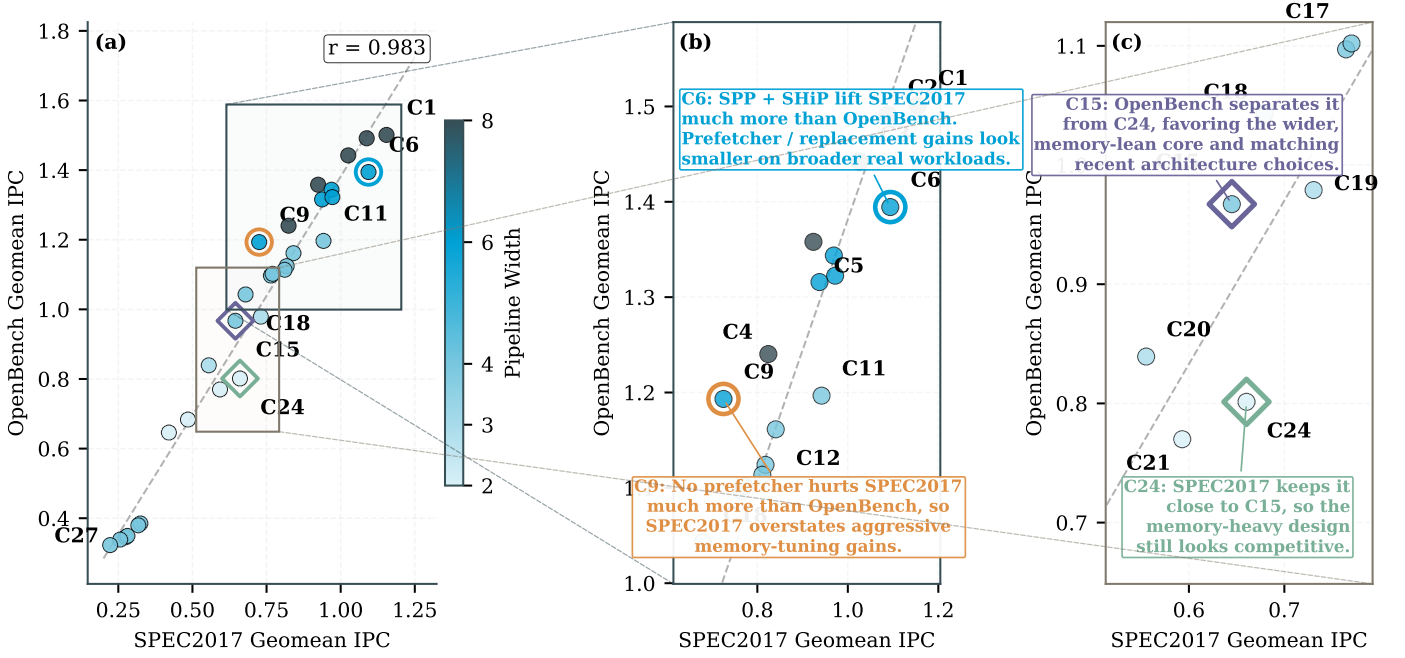


Fig. 5. Geometric-mean IPC of *openbench* versus SPEC2017 across 30 microarchitectural configurations (C1–C30), colored by pipeline width. (a) Full scatter plot showing the strong linear correlation ($r = 0.983$), while notable outliers reveal configuration-specific divergences invisible to either suite alone. (b) Zoomed view detailing the two most off-trend configurations. The different responses of C9 and C6 show that aggressive prefetcher and cache-replacement choices appear more effective on SPEC2017 than on *openbench*, suggesting that these memory optimizations are partly overfit to SPEC2017 and deliver smaller gains on broader modern workloads. (c) Zoomed view of C24 (2-wide, memory-heavy: deep ROB, 1 MB L2, SPP prefetcher) versus C15 (4-wide, memory-lean: shallow ROB, 256 kB L2, no prefetcher). SPEC2017 treats the two designs as nearly tied, whereas *openbench* separates them and favors C15, revealing a design distinction that SPEC2017 alone obscures and making *openbench* more informative for architecture evaluation and selection.

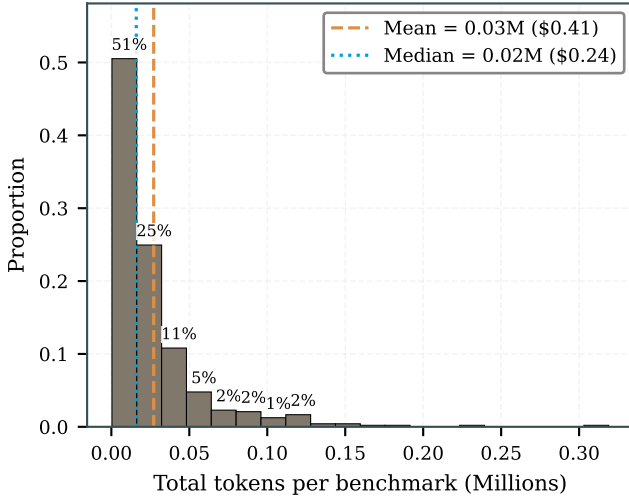


Fig. 6. Token-usage distribution of *openbench-universe* under MiniMax-M2.5 in millions of tokens. Most benchmarks fall below 0.05M tokens. The mean token usage is 0.03M and the median token usage is 0.021M, showing the extremely low cost and scalability of *BenchmarkAnything*.

Performance fidelity. Figure 9 overlays normalized geometric-mean IPC curves for the core subsets and the full universe. Core-8 achieves a mean absolute error of 3.14%, a p_{90} error of 6.54%, and a maximum error of 9.10%. For a direct, equal-cardinality comparison with the 23 SPEC CPU2017 benchmarks, we also construct Core-23 as a *nested superset* of Core-8: we retain all eight Core-8 medoids and greedily add fifteen further benchmarks, each chosen to most reduce the residual error against the full

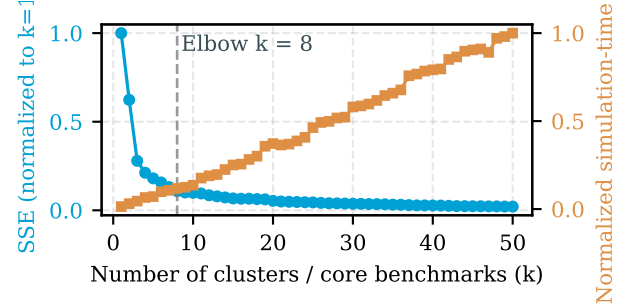


Fig. 7. Determining the optimal number of core benchmarks (k). The left axis denotes the SSE normalized to $k = 1$. The right axis displays a normalized simulation-cost proxy computed by summing the selected medoids' *simSeconds* measured under baseline configuration C30 and then normalizing by the largest evaluated subset size. K-medoids clustering in the PCA space reveals an elbow at $k = 8$, offering a strong trade-off between representative accuracy and simulation efficiency.

universe. Because $\text{Core-8} \subset \text{Core-23}$, every set-monotone coverage measure (including the convex hull of Sec. IV) is guaranteed not to shrink as the subset grows, removing the counterintuitive possibility that a smaller subset covers more of the behavioral space than a larger one. Core-23 tightens these bounds to 0.83%, 2.06%, and 2.31%, respectively. The lower panel further shows that the Core-23 residual bars are smaller than Core-8's for most configurations and lower in aggregate, confirming that the larger subset preserves configuration-specific behavior more faithfully rather than only matching the average trend.

Finally, we compare *openbench-core* against SPEC2017 in the shared t-SNE embedding. As shown in Figure 4(b) and (c), Core-8 achieves $2.12\times$ the SPEC2017 convex-hull

TABLE III
openbench-core CORE-8 BENCHMARK SET SELECTED BY
 PCA+K-MEDOIDS. EACH MEDOID REPRESENTS ONE BEHAVIORAL
 CLUSTER AND IS CHOSEN TO MAXIMIZE COVERAGE.

ID	Benchmark	Domain	Justification
1	mlc-like	System / Embedded	Memory latency stress test
2	hashbrown	Data Structures	Rust std HashMap, hash-intensive
3	stb-bench	Media / Signal	Single-header image codec
4	mbedtls	Cryptography	Widely-deployed TLS library
5	box2d-v3	Graphics / Sim	Industry-standard physics engine
6	minimap2	Bioinformatics	Reference genome aligner
7	Fairy-Stockfish	AI / Game	Chess variant engine, tree search
8	HPCCG	Numerical / HPC	DOE conjugate-gradient proxy app

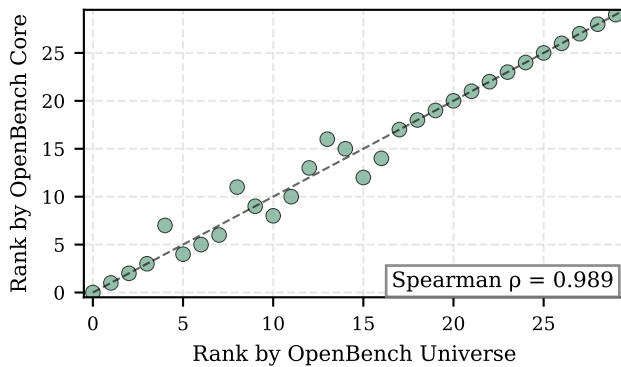


Fig. 8. Correlation between configuration rankings. The x-axis represents the rank by the entire *openbench-universe*. The y-axis represents the rank by the *openbench-core* subset. Both rankings evaluate the geometric mean of IPC with the best configuration ranked as 0. The evaluation yields a Spearman $\rho = 0.989$. This correlation confirms that hardware design decisions made using the 8-benchmark core subset closely mirror those made using the full benchmark suite.

area; because Core-23 is a nested superset of Core-8, its hull necessarily contains the Core-8 hull and therefore covers at least as much of the behavioral space, reaching $2.13\times$. Even small distilled subsets thus preserve much of the broad microarchitectural diversity of the full universe while dramatically reducing simulation cost.

V. DISCUSSION

Our analysis reveals a recurring pattern across a broad set of microarchitectural structures. As shown in Figure 10, optimizations that appear highly effective on SPEC2017 often generalize less reliably to the more diverse *openbench-universe*. Every panel uses the same visual grammar: a cumulative distribution over the workloads of each suite with dashed per-suite median lines. Panel a shows that the two suites agree on the median width speedup near 2.0, but only *openbench* exposes a strongly width-sensitive tail, with 47 of 272 workloads above the SPEC2017 maximum. Panel b shows that prefetcher and replacement gains are consistently positive on SPEC2017 while the *openbench* median sits near zero: returns measured on the legacy suite generalize to fewer

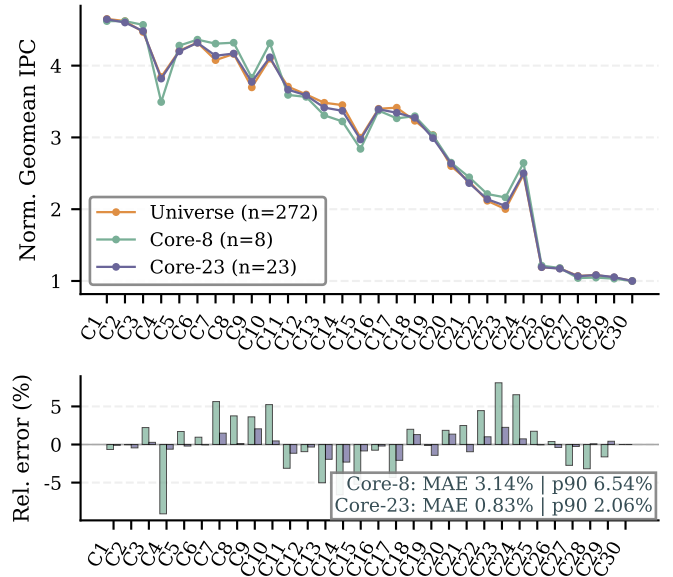


Fig. 9. Performance fidelity of the core subsets compared to the *openbench-universe* across 30 configurations (C1–C30). The top panel overlays the normalized geometric-mean IPC trends for the full universe, Core-8, and Core-23. The bottom panel reports each subset’s relative error versus the full universe. Core-8 achieves mean absolute / p90 / max errors of 3.14% / 6.54% / 9.10%, while Core-23 tightens these bounds to 0.83% / 2.06% / 2.31%.

modern workloads. Panels c to e move beyond IPC and report hardware event rates: D-cache misses per kilo instruction, L2 miss rate as a proxy for memory pressure and working set size, and branch misprediction rate.

These event distributions expose exactly the information an IPC-only view can hide: *openbench* extends the envelope on D-cache MPKI, L2 miss rate, and branch misprediction MPKI, with 39 workloads exceeding the SPEC2017 maximum L2 miss rate and 12 exceeding its maximum branch misprediction MPKI. *openbench* thus broadens memory-hierarchy and control-flow coverage rather than uniformly shifting every metric, reflecting its mix of compact compute kernels and a heavy memory-bound tail, so the two suites are complementary rather than interchangeable.

These discrepancies suggest that contemporary CPU microarchitectures may be partially over-tuned to legacy benchmark behavior. For decades, the community has used SPEC suites as a primary metric for academic and industrial success, so branch predictors, cache replacement policies, and data prefetchers may have been disproportionately optimized for the regular access patterns that dominate these legacy workloads. This creates a feedback loop in which hardware mechanisms are designed to maximize SPEC performance and are then validated on the same suite. The diminished returns observed on *openbench* suggest that some highly regarded optimizations generalize less reliably to the heterogeneous execution behaviors of modern software ecosystems.

We therefore raise an important question for the architecture community: how much of our established microarchitectural intuition reflects general performance truth, and how much reflects historical benchmarking bias? Future design studies

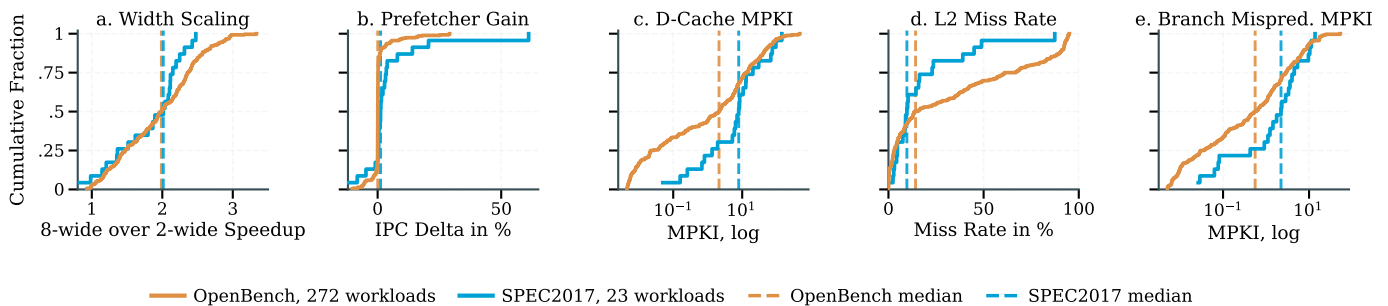


Fig. 10. Microarchitectural characteristics of *openbench* and SPEC CPU2017. Every panel shows one cumulative distribution per suite, and dashed vertical lines mark per-suite medians. Panel a reports the IPC speedup from the 2-wide C24 to the 8-wide C1: the medians agree near 2.0, yet 47 of 272 *openbench* workloads exceed the SPEC2017 maximum of 2.5. Panel b reports the IPC delta between SPP with DRRIP in C1 and a stride prefetcher with SHiP in C2: returns are consistently positive on SPEC2017 but near zero for the median *openbench* workload, so prefetching gains measured on SPEC2017 overstate what broader workloads receive. Panels c to e report hardware event rates on the mainstream reference core C12, recomputed from the same gem5 runs: *openbench* extends the data-side envelope, with a D-cache MPKI tail reaching 470 versus 141 for SPEC2017, an L2 miss rate tail reaching 95% with 39 workloads above the SPEC2017 maximum, and a branch misprediction MPKI tail reaching 53 versus 14. The two suites are complementary rather than interchangeable.

should complement legacy suites with broader and continuously refreshed workload collections, a direction that agent-driven frameworks such as *BenchmarkAnything* make practical through large-scale workload acquisition, standardization, and distillation.

Our current evaluation also has clear bounds. A primary limitation is the use of gem5 in syscall-emulation mode, which abstracts away kernel execution, page faults, context switches, and complex input-output behavior. Workloads that depend heavily on operating-system activity or network stacks may therefore not expose their full bottlenecks in the current setup.

These bounds are properties of the *current instance* of OpenBench, not of the *BenchmarkAnything* methodology. Today’s instance targets single-threaded, user-level, statically linked, AVX-free x86 binaries because that is the envelope gem5 SE mode accepts; this scope prunes some application classes, such as multithreaded, network-intensive, or OS-intensive software and AVX-512-heavy kernels, introducing a degree of selection bias. Every such constraint, however, lives in the build- and validation-rule set given to the agents, not in the agent flow itself. Retargeting the same loop to gem5 full-system mode, ARM or RISC-V, AVX-enabled rebuilds, or multithreaded workloads requires changing those declarative constraints rather than redesigning the framework; the methodology is portable even though the present artifact is deliberately scoped for reproducibility.

Future work will extend *BenchmarkAnything* in three directions: full-system simulation so that agent-generated workloads capture richer operating-system behavior, agent workflows that synthesize and compile multi-threaded applications, and heterogeneous targets including GPUs and domain-specific accelerators. Together, these extensions will broaden *openbench-universe* into a more comprehensive evaluation platform for future system architectures.

VI. CONCLUSION

Evaluating modern processors requires workloads that accurately reflect contemporary software. To overcome the limitations of manual benchmark curation, we presented *BenchmarkAnything*, an automated agent-driven methodology. This framework synthesizes *openbench-universe*, a suite of 272

applications collected from open-source platforms, and distills it into *openbench-core*, a highly efficient representative subset. Our evaluation demonstrates that Core-8 in *openbench-core* achieves up to $2.12\times$ the behavioral-space coverage of SPEC CPU2017 while preserving strong ranking and performance fidelity to the full universe. By reducing human selection bias and exposing critical execution bottlenecks missed by legacy suites, *openbench* provides a compelling target for designing more robust future architectures.

REFERENCES

- [1] M. Hassan, C. H. Park, and D. Black-Schaffer, “A reusable characterization of the memory system behavior of spec2017 and spec2006,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 18, pp. 1 – 20, 2021.
- [2] A. Navarro-Torres, J. Alastruey-Benedé, P. Ibáñez-Marín, and V. Viñals-Yúfera, “Memory hierarchy characterization of spec cpu2006 and spec cpu2017 on the intel xeon skylake-sp,” *PLoS ONE*, vol. 14, no. 8, p. e0220135, 2019.
- [3] L. Wang, J. Zhan, C. Luo, Y. Zhu, Q. Yang, Y. He, W. Gao, Z. Jia, Y. Shi, S. Zhang, C. Zheng, G. Lu, K. Zhan, X. Li, and B. Qiu, “Bigdatabench: A big data benchmark suite from internet services,” in *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. Piscataway, NJ, USA: IEEE, 2014, pp. 488–499.
- [4] Y. Wang, V. Lee, G.-Y. Wei, and D. Brooks, “Predicting new workload or cpu performance by analyzing public datasets,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 15, pp. 1 – 21, 2019.
- [5] Z. Jia, L. Wang, J. Zhan, L. Zhang, and C. Luo, “Characterizing data analysis workloads in data centers,” in *2013 IEEE International Symposium on Workload Characterization (IISWC)*. Piscataway, NJ, USA: IEEE, 2013, pp. 66–76.
- [6] Y. Wang, S. Furman, N. Hardy, M. Ellis, G. Back, Y. Hong, and K. Cameron, “A detailed historical and statistical analysis of the influence of hardware artifacts on spec integer benchmark performance,” *IEEE Transactions on Computers*, vol. 73, pp. 1262–1274, 2024.
- [7] C. Buduleci, A. Gellert, A. Florea, and R. Brad, “Architectural and technological approaches for efficient energy management in multicore processors,” *Comput.*, vol. 13, p. 84, 2024.
- [8] S. Kounev, K. Lange, and J. V. Kistowski, “The spec cpu benchmark suite,” in *Systems Benchmarking*. Cham, Switzerland: Springer, 2020, pp. 231–250.
- [9] S. Singh and M. Awasthi, “Memory centric characterization and analysis of spec cpu2017 suite,” in *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*. New York, NY, USA: ACM, 2019, pp. 285–295.
- [10] A. Limaye and T. Adegbiya, “A workload characterization of the spec cpu2017 benchmark suite,” in *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Piscataway, NJ, USA: IEEE, 2018, pp. 149–158.

- [11] R. Panda, S. Song, J. Dean, and L. John, "Wait of a decade: Did spec cpu 2017 broaden the performance horizon?" in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. Piscataway, NJ, USA: IEEE, 2018, pp. 271–282.
- [12] S. Song, Q. Wu, S. Flolid, J. Dean, R. Panda, and J. Deng, "Experiments with spec cpu 2017: Similarity, balance, phase behavior and simpoinits," 2018.
- [13] J. N. Amaral, E. Borin, D. R. Ashley, C. Benedicto, E. Colp, J. H. S. Hoffmam, M. Karpoff, E. Ochoa, M. Redshaw, and R. E. Rodrigues, "The alberta workloads for the spec cpu 2017 benchmark suite," in *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Piscataway, NJ, USA: IEEE, 2018, pp. 159–168.
- [14] C. Wang, L. Wang, W. Gao, Y. Yang, Y. Zhou, and J. Zhan, "Achieving consistent and comparable cpu evaluation outcomes," arXiv:2411.08494, 2024.
- [15] T. Toprak, M. Loibl, G. Teixeira, I. Shiskina, C. Miao, J. Kiendl, B. Marussig, and F. Kummer, "Employing continuous integration inspired workflows for benchmarking of scientific software - a use case on numerical cut cell quadrature," arXiv:2503.17192, 2025.
- [16] I. Mallona, A. Luetge, C. Soneson, B. Carrillo, R. Gerber, D. Incicau, A. Sonrel, and M. D. Robinson, "Building a continuous benchmarking ecosystem in bioinformatics," *PLOS Computational Biology*, vol. 21, p. e1013658, 2024.
- [17] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, pp. 1 – 30, 2024.
- [18] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," arXiv:2409.02977, 2024.
- [19] H. Jin, L. Huang, H. Cai, J. Yan, B. Li, and H. Chen, "From llms to llm-based agents for software engineering: A survey of current, challenges and future," arXiv:2408.02479, 2024.
- [20] Y. Wang, W. Zhong, Y. Huang, E. Shi, M. Yang, J. Chen, H. Li, Y. Ma, Q. Wang, and Z. Zheng, "Agents in software engineering: survey, landscape, and vision," *Automated Software Engineering*, vol. 32, no. 2, p. 70, 2025.
- [21] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, and J. C. Grundy, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, pp. 1 – 79, 2023.
- [22] Y. Cheng, C. Zhang, Z. Zhang, X. Meng, S. Hong, W. Li, Z. Wang, Z. Wang, F. Yin, J. Zhao, and X. He, "Exploring large language model based intelligent agents: Definitions, methods, and prospects," arXiv:2401.03428, 2024.
- [23] L. Applis, Y. Zhang, S. Liang, N. Jiang, L. Tan, and A. Roychoudhury, "Unified software engineering agent as ai software engineer," arXiv:2506.14683, 2025.
- [24] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2024, pp. 1592–1604.
- [25] Z. Zheng, K.-C. Ning, J. Chen, Y. Wang, W. Chen, L. Guo, and W. Wang, "Towards an understanding of large language models in software engineering tasks," *Empirical Software Engineering*, vol. 30, no. 2, p. 50, 2025.
- [26] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Transactions on Software Engineering*, vol. 50, pp. 911–936, 2023.
- [27] S. I. Ross, F. Martinez, S. Houde, M. J. Muller, and J. D. Weisz, "The programmer's assistant: Conversational interaction with a large language model for software development," in *Proceedings of the 28th International Conference on Intelligent User Interfaces*. New York, NY, USA: ACM, 2023, pp. 491–514.
- [28] T. Weber, M. Brandmaier, A. Schmidt, and S. Mayer, "Significant productivity gains through programming with large language models," *Proceedings of the ACM on Human-Computer Interaction*, vol. 8, pp. 1 – 29, 2024.
- [29] B. Radenkovic, S. P. Prokhorov, M. Radenković, and A. Labus, "Application of large language models in software development: Review of the current state and development perspectives," in *2025 6th International Workshop on Engineering Technologies and Computer Science (EnT)*. Piscataway, NJ, USA: IEEE, 2025, pp. 1–7.
- [30] HKUDS, "Cli-anything: Making all software agent-native," <https://github.com/HKUDS/CLI-Anything>, 2026, gitHub repository, accessed 2026-04-03.
- [31] G. Huntley, "Ralph wiggum as a "software engineer": An agentic coding loop," <https://ghuntley.com/loop/>, 2025, accessed: 2026-06-04.
- [32] S. Ainsworth and T. M. Jones, "MuonTrap: Preventing cross-domain spectre-like attacks by capturing speculative state," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*. Piscataway, NJ, USA: IEEE, 2020, pp. 132–144.
- [33] K. Hoste and L. Eeckhout, "Microarchitecture-independent workload characterization," *IEEE Micro*, vol. 27, no. 3, pp. 63–72, 2007.
- [34] J. J. Yi, D. J. Lilja, and D. M. Hawkins, "A statistically rigorous approach for improving simulation methodology," in *Proceedings of the 9th International Symposium on High-Performance Computer Architecture (HPCA)*. Piscataway, NJ, USA: IEEE, 2003, pp. 281–291.
- [35] A. Phansalkar, A. Joshi, and L. K. John, "Analysis of redundancy and application balance in the SPEC CPU2006 benchmark suite," in *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*. New York, NY, USA: ACM, 2007, pp. 412–423.